Agentic Design Patterns

Comprehensive Visual Summary

Based on the book by Antonio Gulli · Google Engineer · Springer 2025

21	4	424	3
Design Patterns	Parts	Pages	Frameworks

All royalties from the original book are donated to Save the Children

What Is an AI Agent?

An **AI agent** is a system designed to perceive its environment and take actions to achieve a specific goal. It is a significant evolution beyond a standard Large Language Model (LLM): while an LLM can answer questions from its training data, an agent can **plan**, **use tools**, and **interact with the world** over multiple steps. Think of an agent as a smart assistant that works through a continuous loop: it receives a mission, gathers context, devises a plan, executes actions, then learns from the outcome.

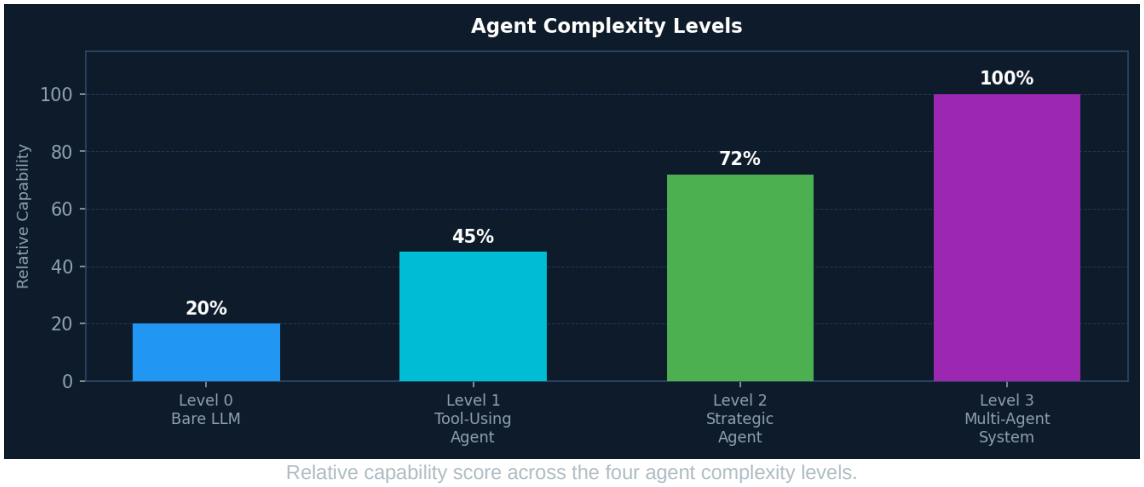
The Five-Step Agentic Loop

1. Get the Mission	The user (or an orchestrator) provides a high-level goal, e.g. "organise my schedule."
2. Scan the Scene	The agent gathers context — reads emails, checks calendars, queries databases.
3. Think It Through	It devises a plan: what sequence of actions will best achieve the goal?
4. Take Action	It executes: sends invites, updates records, calls APIs, delegates to sub-agents.
5. Learn & Iterate	It observes outcomes, self-corrects, and refines its approach for future tasks.

Agent Complexity Levels (Level 0 → Level 3)

Agents are not a single monolithic concept. The book defines four tiers of complexity, each unlocking new capabilities and new design patterns:

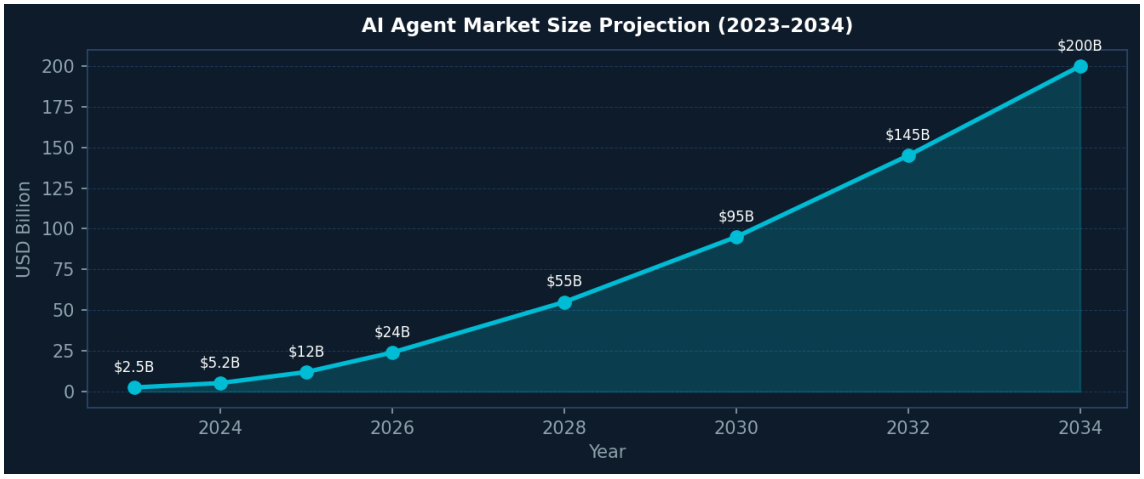
Level 0 — Bare LLM	Operates solely from pre-trained knowledge. No tools, no memory, no environment access. Strong for conceptual explanation, but blind to current events.
Level 1 — Tool-Using Agent	Connects to external tools (web search, databases, APIs). Can fetch real-time data, call financial APIs, run RAG queries. First true agentic tier.
Level 2 — Strategic Agent	Multi-step planning, context engineering, self-improvement. Manages full workflows (e.g. bug reports → fix → test → submit). Proactive and adaptive.
Level 3 — Multi-Agent System	A team of specialised agents coordinated by an orchestrator. Mirrors human organisation: division of labour, parallel execution, inter-agent communication.



Relative capability score across the four agent complexity levels.

Market Context

AI agent adoption is accelerating rapidly. By end of 2024, agent startups had raised over $2 billion and the market was valued at $5.2 billion. Industry projections estimate this will explode to **nearly $200 billion by 2034**. A majority of large IT companies are already deploying agents, with a fifth having started in the past year alone.



AI agent market size projection 2023–2034 (illustrative trend from industry reports).

The Evolution: From LLM to Agentic AI

In just two years, the AI paradigm shifted through four distinct eras:

Era 1 — Basic LLM Prompting	Simple prompt → LLM → response. No memory, no tools. Useful but brittle.
Era 2 — RAG	Retrieval-Augmented Generation grounds the LLM in external, factual documents, dramatically reducing hallucination for domain-specific tasks.
Era 3 — Individual AI Agents	Agents equipped with tools can act: call APIs, write files, run code, query DBs. Single-agent systems become productive autonomous workers.
Era 4 — Agentic AI (Multi-Agent)	Teams of specialised agents collaborate, delegate, and communicate. Complex business workflows can be automated end-to-end with minimal human intervention.

Why Design Patterns? Just as software engineering patterns (GoF) gave developers a shared vocabulary for object-oriented problems, **agentic design patterns** provide reusable, battle-tested blueprints for the recurring challenges of building intelligent agent systems: orchestration, memory, tool integration, safety, and coordination.

Frameworks Used in This Book

All 21 chapters provide hands-on code examples for one or more of three frameworks:

LangChain / LangGraph	Highly flexible Python library for chaining LLM calls. LangGraph extends it with stateful, graph-based workflows supporting conditional branches and loops. Ideal for complex sequential and reflective pipelines. Largest ecosystem.
Crew AI	Framework purpose-built for multi-agent orchestration. Defines agents by role, goal, and backstory; assigns tasks; manages delegation. Excellent for collaborative team-of-agents architectures.

Google ADK (Agent Developer Kit)	Google's open-source toolkit for building, evaluating, and deploying agents. Native integration with Gemini models and Vertex AI. Provides Session/State/Memory services and a structured event-driven runtime.

PART ONE — Core Orchestration Patterns (Chapters 1–7)

The seven core patterns form the foundation of every agentic system. They address the most fundamental design questions: how to structure the flow of LLM calls, how to make decisions, how to exploit parallelism, how to self-improve, how to interact with the world, how to plan ahead, and how to coordinate teams.

Ch 1 Prompt Chaining

Prompt chaining, also called the Pipeline pattern, addresses a fundamental limitation: asking a single LLM call to solve a complex, multi-faceted problem often leads to poor results. The model struggles with instruction overload, loses context mid-response, and error in one part corrupts the rest. The solution is **divide and conquer**: break the task into a sequence of smaller, focused sub-problems, each handled by a dedicated prompt, with the output of each step feeding as input into the next.

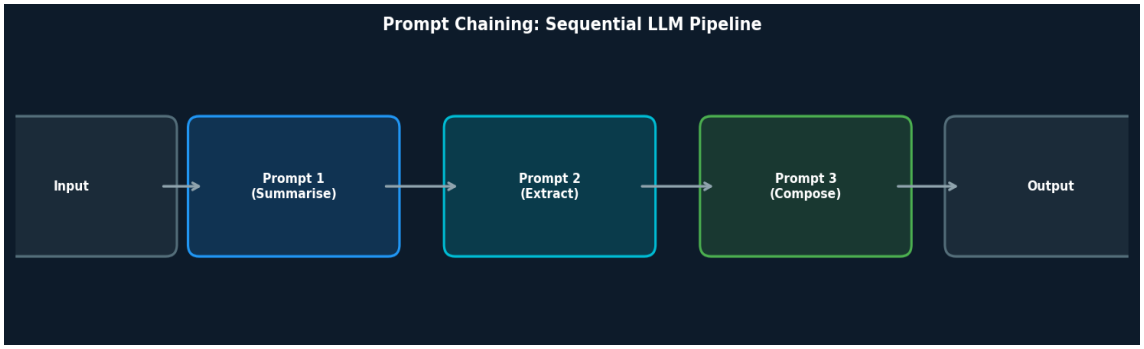
Why Single Prompts Fail for Complex Tasks

Consider the request: "Analyse this market research report, summarise findings, identify the top three trends with data points, and draft an email to the marketing team." Given to a single prompt, the model might summarise well but fail to extract data or draft the email properly. The cognitive load is too high, and early errors propagate forward. Problems specific to single-prompt monoliths include:

- **Instruction neglect** — parts of a long, complex prompt are silently ignored.
- **Contextual drift** — the model loses track of earlier requirements.
- **Error propagation** — a mistake in step 1 corrupts all subsequent reasoning.
- **Context window pressure** — long inputs leave less room for quality output.
- **Hallucination risk** — higher cognitive load increases fabrication.

How Prompt Chaining Solves This

By decomposing the task above into three focused chains — (1) Summarise, (2) Extract trends, (3) Draft email — each LLM call has a single, clear responsibility. The output of step 1 becomes the input of step 2, and so on. Each step can even be assigned a distinct **role** (e.g. "Market Analyst", "Trade Analyst", "Expert Documentation Writer") to further sharpen focus.



A three-step prompt chain: each specialised LLM call produces structured output for the next.

Structured Output Between Steps

The reliability of a chain depends entirely on the integrity of data passed between steps. If the output of one prompt is ambiguous or free-form, the next prompt may misinterpret it. The solution is to mandate **structured output formats** (JSON, XML) so that data is machine-readable and precisely parseable. For example, the trend extraction step might return a JSON array of objects with *trend_name* and *supporting_data* fields, which the

email-drafting step can reliably consume.

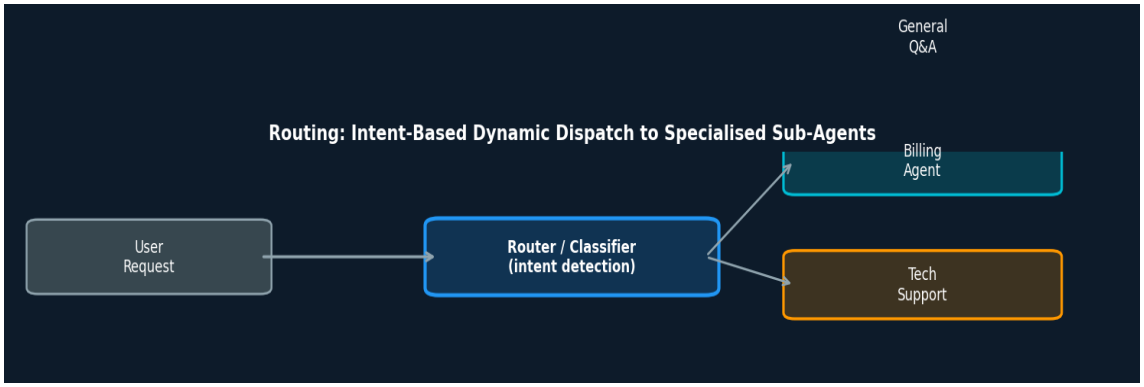
When to use	Sequential, multi-stage tasks where each step depends on the prior one. Document processing, report generation, multi-transform pipelines.
Key benefit	Modularity, debuggability, and reduced cognitive load per LLM call.
Key challenge	Brittle handoffs: if one step produces unexpected output format, the chain breaks. Invest in structured output schemas.
Frameworks	LangChain LCEL (pipe operator `|`), LangGraph (nodes/edges), Google ADK (SequentialAgent).

Ch 2 Routing

Sequential prompt chaining handles linear workflows well, but real-world systems must make **dynamic decisions**: depending on what the user asks, different expertise or different processing paths are needed. The **Routing** pattern introduces a classifier or decision-making component that analyses incoming requests and dispatches them to the most appropriate handler, sub-agent, or processing path.

How Routing Works

The core of this pattern is a **Router** — typically a fast, lightweight LLM call or a rule-based classifier — that determines intent. Based on this classification, the request is forwarded to one of several specialised sub-systems. For example, a customer support system might route to a Technical Support agent, a Billing agent, or a General Q&A; agent based on the customer's query.



Routing: the classifier dispatches requests to three specialised sub-agents.

Forms of Routing

- **LLM-based classification**: The router calls a small LLM with a classification prompt, returning a label (e.g. "technical", "billing", "general"). Low latency if using a small model.
- **Rule-based routing**: Deterministic keyword or regex matching for high-confidence, low-cost cases.
- **Embedding similarity**: Compute the query embedding and compare to cluster centroids to determine the closest category.
- **Hierarchical routing**: A parent agent routes to a child coordinator, which routes further. Useful for large, complex systems.

When to use	Systems serving multiple user intents (e.g. a support bot handling tech, billing, HR, logistics). Multi-domain Q&A.; Content moderation pipelines.
Key benefit	Specialisation: each sub-agent can be optimised for its domain, reducing cost and improving accuracy.

Key challenge	Misclassification: an incorrect route can give the user a poor experience. Use confidence thresholds and fallback paths.
Frameworks	LangChain RunnableBranch, Google ADK auto-flow delegation, Crew AI manager agents.

Ch 3 Parallelization

Many complex tasks contain independent sub-tasks that need not be executed sequentially. The **Parallelization** pattern runs multiple LLM calls or tool invocations **concurrently**, then aggregates their results. This dramatically reduces total latency — instead of running N steps sequentially (sum of latencies), all N run in parallel (max of latencies).

When Tasks Can Be Parallelised

The prerequisite for parallelisation is **independence**: sub-tasks must not depend on each other's outputs. For example, researching "renewable energy", "electric vehicles", and "carbon capture" for a sustainability report are three completely independent research tasks that can run simultaneously. A synthesis agent then combines all three outputs into the final report.

Fan-out / Fan-in	Multiple independent LLM calls run in parallel (fan-out), results combined by a synthesis agent (fan-in). The canonical parallelisation pattern.
Voting / Ensemble	The same question is sent to multiple agents with different prompts or temperatures. Results are compared or voted on to improve accuracy and reduce hallucination.
Batch Processing	Parallelize over a list of inputs (e.g. score 100 support tickets simultaneously) for throughput.
Async Tool Calls	Multiple external tool calls (search, database, API) fire simultaneously rather than sequentially.
When to use	Multi-perspective research, batch evaluation, independent sub-task decomposition.
Key benefit	Latency reduction (wall-clock time = max, not sum) and throughput gains.
Key challenge	Aggregation complexity: merging parallel outputs coherently. Dependency detection: must ensure truly independent tasks.
Frameworks	LangChain RunnableParallel, LangGraph parallel nodes, Google ADK ParallelAgent + SequentialAgent.

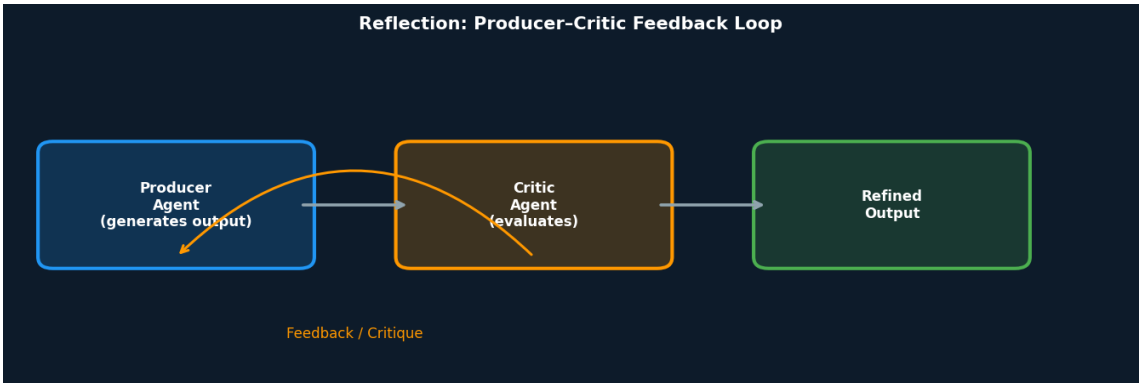
Ch 4 Reflection

Even with well-structured workflows, an agent's first output may not be optimal. The **Reflection** pattern introduces a **feedback loop**: the agent evaluates its own output — or delegates evaluation to a specialised Critic agent — identifies weaknesses, and iteratively refines the result. This transforms agents from one-shot executors into self-improving systems.

The Producer–Critic Model

The most effective implementation separates reflection into two distinct roles. The **Producer Agent** focuses entirely on generating output (code, essay, plan). The **Critic Agent** is given a distinct persona and mandate — "You are a senior software engineer reviewing this code for correctness and security" — and evaluates the

Producer's work against specific criteria. Its critique is fed back to the Producer, which generates a refined version. This separation prevents the "cognitive bias" that emerges when an agent reviews its own work.



Reflection: the Critic provides structured feedback that the Producer uses to iterate.

The Reflection Loop Steps

1. Execute	Producer performs the task and generates initial output.
2. Evaluate/Critique	Critic analyses: factual accuracy, coherence, style, completeness, adherence to constraints.
3. Reflect/Refine	Based on critique, Producer generates an improved version, or the plan is adjusted.
4. Iterate	Steps 1–3 repeat until a stopping condition is met (quality threshold, max iterations, or user acceptance).

Synergy With Other Patterns

Reflection is most powerful in combination with other patterns. With **Goal Setting & Monitoring** (Ch 11), the goal provides the benchmark for self-evaluation. With **Memory** (Ch 8), the agent learns from past critiques, building cumulative improvement rather than isolated corrections. Without memory, each reflection cycle is stateless — the agent cannot avoid repeating the same mistakes across sessions.

When to use	Creative writing, code generation, plan validation, essay revision — any task where quality matters more than speed.
Key benefit	Self-correction: higher-quality outputs without human review at every step.
Key challenge	Termination: without a clear stopping criterion, reflection loops can run indefinitely. Define max iterations or a quality score threshold.
Frameworks	LangGraph (cycles/loops), Google ADK (LoopAgent), Crew AI (process=iterative).

Ch 5 Tool Use / Function Calling

An LLM's knowledge is frozen at training time. To interact with the real world — get live data, write to databases, send emails, run code — agents need **Tool Use**, implemented via **Function Calling**. The LLM is presented with a menu of available tools (functions described in structured JSON schemas), decides which tools to call based on the user's request, generates structured arguments, and the framework executes the actual function. The result is fed back to the LLM as context for the next step.

The Tool Use Lifecycle

1. Tool Definition	Developer describes each tool: name, purpose, parameters (names, types, descriptions). Exposed to the LLM via the API.
2. LLM Decision	The LLM reads the user request and available tools. It decides whether to use a tool, which one, and with what arguments.
3. Call Generation	LLM outputs a structured JSON object: {"tool": "get_weather", "args": {"city": "London"}}.
4. Tool Execution	The agentic framework intercepts this output, calls the actual function with the given arguments.
5. Observation	The tool's return value is injected back into the LLM's context as a new "observation" message.
6. LLM Processing	The LLM uses the tool result to formulate the final response or decide the next action (possibly calling another tool).

Tool Use vs. MCP

Function calling is a **direct, one-to-one** mechanism: the LLM calls a specific pre-defined function registered with the framework. It is proprietary and varies across providers. The **Model Context Protocol (MCP)**, covered in Chapter 10, provides a **standardised, open protocol** that allows any compliant LLM to discover and use any compliant tool without custom integration. MCP is the evolution of function calling toward a universal ecosystem.

When to use	Any time the agent needs live data, computation, or real-world actions: weather, stock prices, database CRUD, email, calendar, code execution.
Key benefit	Breaks the LLM's training-data limitations. Enables real-world agency.
Key challenge	Security and trust: a tool with broad permissions (delete files, send emails) must be guarded against misuse. Apply least-privilege principles.
Frameworks	LangChain @tool decorator, Google ADK FunctionTool/agent_tool, Crew AI @tool, OpenAI tools parameter.

Ch 6 Planning

When a task requires foresight — multiple steps, dependencies, contingencies — reactive tool-calling is not enough. The **Planning** pattern gives agents the ability to **formulate a multi-step action plan before executing**. Rather than being told the "how", the agent autonomously discovers the optimal sequence of actions to move from an initial state to a goal state.

Dynamic Planning vs Fixed Workflows

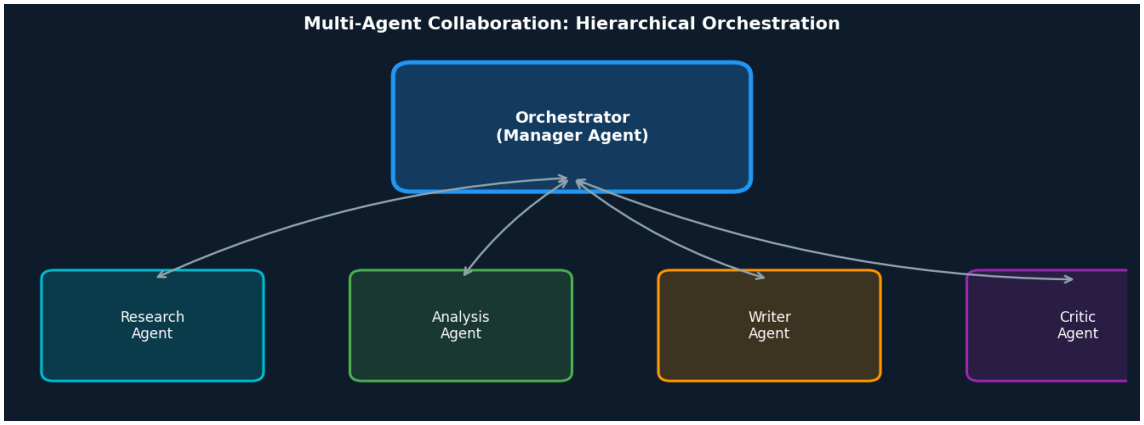
A critical design decision is when to plan dynamically vs when to use a fixed, predetermined workflow. Dynamic planning is powerful but introduces uncertainty — the agent might choose unexpected paths. When the solution is already known and repeatable (e.g. always the same 5-step onboarding flow), a fixed workflow is safer, more predictable, and easier to audit. **The key question: does the "how" need to be discovered, or is it already known?**

> **Example — Adaptive Planning:** "Organise a team offsite for 30 people in Lisbon next quarter." The agent must discover: budget, venue options, caterer, travel logistics, agenda. If the preferred venue is fully booked, a capable planning agent registers this constraint and re-plans — it doesn't simply fail.

ReAct Planning	Reason + Act: the agent interleaves reasoning ("I need to find venues") with tool calls ("search_venues(city=Lisbon)"). Popular for open-ended tasks.
Plan-then-Execute	The agent first produces a complete plan, then executes each step. Useful when human review of the plan before execution is desired.
Hierarchical Plan	A top-level plan is decomposed into sub-plans assigned to sub-agents. Used in complex multi-agent systems (connects to Ch 7).
When to use	Complex, multi-step goals with contingencies: project management, travel booking, autonomous coding, business workflow automation.
Key benefit	Goal-oriented behaviour: the agent can handle novel situations and adapt its path when circumstances change.
Key challenge	Predictability and control: dynamic plans can take unexpected paths. Combine with HITL (Ch 13) and Guardrails (Ch 18) for high-stakes tasks.
Frameworks	LangGraph (graph traversal), Google ADK (LlmAgent with planning), Crew AI (hierarchical process).

Ch 7 Multi-Agent Collaboration

When a single agent — however capable — cannot efficiently handle all aspects of a complex, multi-domain task, the answer is **Multi-Agent Collaboration**. This pattern structures the system as a cooperative ensemble of distinct, specialised agents, each owning a specific domain, coordinated by an **Orchestrator** (Manager Agent). The collective performance of the ensemble surpasses what any individual agent could achieve alone.



Hierarchical multi-agent system: orchestrator delegates to four specialised sub-agents.

Collaboration Patterns

Sequential Handoffs	Agent A completes its task and passes output to Agent B. Similar to a prompt chain, but with distinct agent identities and tool sets.
Parallel Processing	Multiple agents work simultaneously on independent sub-tasks; an aggregator synthesises results (connects to Ch 3).
Debate & Consensus	Agents with different perspectives or information sources discuss options and vote on the best answer. Improves robustness.
Hierarchical / Manager	An orchestrator breaks the goal into sub-tasks and delegates to worker agents dynamically. The orchestrator synthesises final output.

Critic-Reviewer	Producer agents create output; a separate Critic group reviews for correctness, safety, and quality before finalisation (connects to Ch 4).
Expert Teams	Agents specialise by domain (researcher, writer, editor, legal reviewer). Each brings specific tools and knowledge.

Key Design Considerations

The power of multi-agent systems comes with engineering complexity. Three critical infrastructure concerns must be solved:

- **Communication Protocol**: Agents need a standardised way to exchange data, delegate tasks, and return results. Frameworks provide this via shared state or message queues.
- **Shared Ontology**: Agents must interpret each other's outputs consistently. Structured schemas prevent ambiguity.
- **Failure Handling**: The failure of one sub-agent should not crash the system. The orchestrator must handle errors gracefully, retry, or escalate.

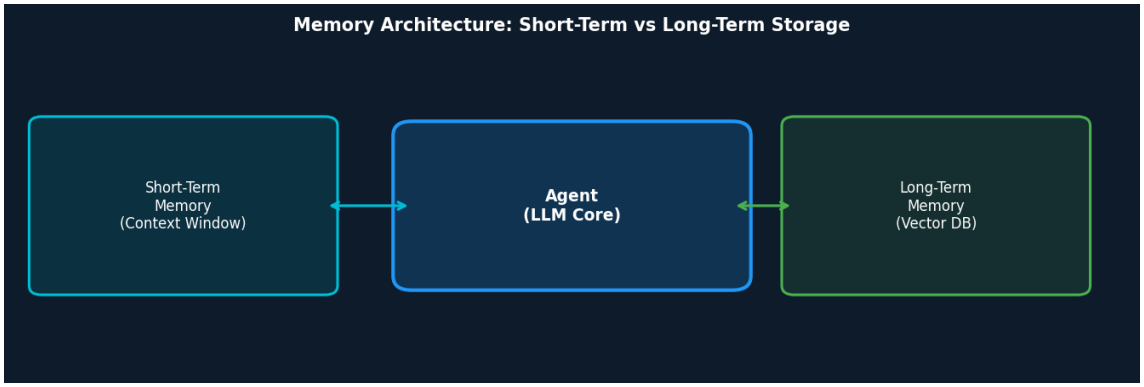
When to use	Complex tasks requiring multiple domains of expertise: product launches, research reports, end-to-end business workflows.
Key benefit	Scalability, specialisation, and fault isolation. The failure of one agent does not cause total system failure.
Key challenge	Communication overhead and emergent complexity. Debugging multi-agent interactions is significantly harder than single-agent flows.
Frameworks	Crew AI (native multi-agent), Google ADK (sub-agents, ParallelAgent), LangGraph (multi-node graphs with agent nodes).

PART TWO — Memory, Learning & Context (Chapters 8–11)

Part Two focuses on the agent's inner life: how it remembers, how it improves over time, how it connects to external resources via standardised protocols, and how it knows when it has succeeded. These patterns transform a reactive system into one that genuinely learns and pursues goals.

Ch 8 Memory Management

An agent with no memory is stateless: every interaction starts from scratch. Real intelligence requires **memory** — the ability to retain and utilise information from past interactions. The **Memory Management** pattern defines two distinct memory types, mirroring human cognitive architecture.



Agent memory architecture: short-term context window and long-term vector database.

Short-Term Memory: The Context Window

Short-term memory is everything in the current context window — recent messages, tool results, agent reasoning, and prior turns in the session. It is fast to access but limited in size and ephemeral: it disappears when the session ends. Modern models with "long context" windows (1M+ tokens) expand this, but even they cannot replace true long-term persistence. Key management techniques:

- **Summarisation**: Compress older parts of the conversation to free space for new context.
- **Selective pruning**: Remove low-relevance messages while keeping critical facts.
- **Context engineering**: Strategically curate what goes into the window at each step (Level 2 skill).

Long-Term Memory: External Persistent Storage

Long-term memory lives outside the agent's LLM, in databases, knowledge graphs, or **vector databases**. When an agent needs information from past sessions, it queries this store, retrieves relevant data via semantic search, and injects it into the current context window. Google ADK models this explicitly with three services: **Session** (one conversation thread), **State** (data within a session), and **Memory** (persistent, cross-session knowledge base).

Use Cases	Personalised assistants (remember preferences), customer support (recall past tickets), coding agents (remember project architecture), learning systems.
Key benefit	Continuity and personalisation: the agent improves with every interaction rather than resetting each time.
Key challenge	Memory staleness and relevance: old memories can become outdated or misleading. Implement TTL (time-to-live) and relevance scoring for retrieval.

Frameworks	Google ADK (SessionService + MemoryService), LangGraph (state persistence), LangChain (vector store memory), Crew AI (agent memory).

Ch 9 Learning and Adaptation

Memory allows agents to remember the past. **Learning** allows them to **improve from it**. The Learning and Adaptation pattern covers the mechanisms by which agents evolve beyond their initial parameters — adjusting their strategies, preferences, and knowledge based on new experience.

Learning Paradigms for Agents

Reinforcement Learning (RL)	Agent takes actions, receives rewards (positive outcomes) or penalties (negative). Learns optimal policies in changing environments. PPO (Proximal Policy Optimisation) is the dominant algorithm: it makes small, safe policy updates within a "trust region" to avoid catastrophic forgetting.
DPO — Direct Preference Optimisation	Specifically designed for aligning LLMs with human preferences. Skips the separate reward model used in RLHF. Instead, directly uses preference data (human rankings) to update the LLM's policy, making the process simpler and more stable.
Few-Shot / Zero-Shot Adaptation	LLM-based agents leverage their pre-trained knowledge to adapt to new tasks with minimal or zero examples. Powerful for rapid deployment in new domains.
Online Learning	Agents update their knowledge continuously as new data arrives — essential for real-time environments like financial trading or live monitoring.
Memory-Based Learning	Agents recall past experiences stored in long-term memory (Ch 8) to adjust behaviour in similar future situations. Complements episodic memory with behavioural adaptation.
When to use	Dynamic environments requiring ongoing adaptation: recommendation engines, fraud detection, autonomous robots, dialogue systems that improve with use.
Key benefit	Agents improve autonomously without constant manual re-training or prompt engineering.
Key challenge	Catastrophic forgetting (new learning overwrites old knowledge) and reward hacking (agents find shortcuts that maximise reward without achieving the real goal).

Ch 10 Model Context Protocol (MCP)

Tool Use (Ch 5) lets agents call functions, but every integration is bespoke. The **Model Context Protocol (MCP)** is an open standard that provides a **universal adapter**: any compliant LLM can connect to any compliant tool, database, or service without custom integration. Think of it as USB-C for AI agents.

MCP Architecture

MCP operates on a **client-server architecture**. An MCP Server exposes three types of capabilities:

- **Resources**: Data sources (files, databases, APIs) the LLM can read.
- **Prompts**: Interactive templates that guide LLM behaviour.
- **Tools**: Actionable functions the LLM can call.

The **MCP Client** (the agent's host application) discovers available capabilities, selects the right ones, and invokes them on behalf of the LLM. This enables a **federated model**: legacy services can be wrapped in an MCP-compliant interface without rewriting, instantly integrating them into modern agentic workflows.

MCP vs Function Calling — Key Differences

Scope	Function calling: direct, one-to-one, proprietary. MCP: open standard, many-to-many, interoperable.
Discovery	Function calling: tools statically defined per agent. MCP: client dynamically discovers available capabilities from any MCP server.
Portability	Function calling: tied to a specific LLM provider's API. MCP: works across Gemini, GPT, Claude, Mixtral, etc.
Ecosystem	MCP enables a plug-and-play marketplace of tools. Developers write an MCP server once; any agent can use it.

Critical Warning: MCP is a contract for an agentic interface, but its effectiveness depends on the quality of the underlying API it wraps. Wrapping a legacy API that returns data in PDF format is useless if the agent can't parse PDF. The underlying data must be agent-friendly (e.g. plain text, Markdown, JSON). Similarly, an API that only returns full records one-by-one will make an agent summarising thousands of records slow and inaccurate. Build APIs with deterministic filtering and sorting to support the non-deterministic agent.

When to use	Any system requiring multiple tools from multiple providers. Enterprise integrations, IDE plugins, tool marketplaces.
Key benefit	Interoperability and reusability: one MCP server, many LLM consumers.
Key challenge	API quality: MCP is only as good as the underlying APIs it exposes. Design APIs for agent consumption.

Ch 11 Goal Setting and Monitoring

An agent without explicit goals is just a reactive system. The **Goal Setting and Monitoring** pattern equips agents with **clear objectives** and the mechanisms to **track progress**, detect deviation, and signal success or failure. It transforms an agent from a task executor into a purposeful system.

Goal Anatomy

Goal Statement	A precise, measurable description of the desired end state. Vague goals lead to vague behaviour.
Success Criteria	Quantifiable thresholds: "Response accuracy > 95%", "Task complete within 10 steps", "Budget under $0.50 per query."
Sub-goals	Decompose complex goals into intermediate milestones the agent can track and achieve incrementally.
Failure Conditions	Explicit conditions that trigger escalation, fallback, or human intervention (connects to Ch 12 and Ch 13).

Monitoring in Practice

Monitoring closes the loop between goal definition and agent behaviour. It involves continuously measuring the agent's progress against success criteria and taking corrective action when drift is detected. The synergy with

Reflection (Ch 4) is direct: monitoring provides the signal, reflection provides the corrective mechanism.

When to use	Any autonomous agent operating over multiple steps or extended time periods. KPI dashboards, SLA compliance, autonomous data pipelines.
Key benefit	Accountability and early failure detection. Agents that know when they're off-track can self-correct before causing downstream harm.
Key challenge	Metric definition: poorly defined success criteria lead to agents that appear to succeed while actually failing the real objective (Goodhart's Law).

PART THREE — Safety, Resilience & Human Oversight (Chapters 12–14)

As agents become more autonomous, the stakes of failure rise. Part Three addresses the essential safety infrastructure: how to handle errors gracefully, when and how to involve humans, and how to ground agent responses in verified facts.

Ch 12 Exception Handling and Recovery

Real-world environments are unpredictable. APIs time out, tools return invalid data, LLMs generate malformed outputs. The **Exception Handling and Recovery** pattern builds resilience into agents: detect failures, recover gracefully, and when recovery is impossible, fail in a controlled and informative way.

Exception Categories

Tool Failures	API timeouts, rate limits, invalid responses from external services. Strategy: exponential backoff, circuit breakers, fallback tools.
LLM Output Errors	Malformed JSON, schema violations, hallucinated function calls. Strategy: output validation, structured output enforcement, retry with corrective prompt.
Planning Failures	Agent's plan is impossible given current world state. Strategy: replan, decompose differently, escalate to human.
Resource Exhaustion	Context window overflow, token budget exceeded. Strategy: summarise context, prune history, switch to a more capable model.
Infinite Loops	Reflection or planning cycles that never converge. Strategy: iteration caps, convergence detection, forced termination.
When to use	Always. Every production agent needs exception handling. It is not optional.
Key benefit	Fault tolerance: a failing sub-agent does not crash the entire system. Controlled degradation preserves user trust.
Key challenge	Distinguishing recoverable from unrecoverable failures quickly, without wasting tokens on doomed retries.

Ch 13 Human-in-the-Loop (HITL)

Full autonomy is not always desirable — or safe. The **Human-in-the-Loop (HITL)** pattern strategically interleaves human judgment with AI execution at critical decision points. Rather than a binary choice between "fully automated" and "fully manual", HITL defines the optimal **collaboration model**: AI handles high-volume, low-stakes decisions autonomously; humans review high-stakes, ambiguous, or novel situations.

When to Involve Humans

High-Stakes Actions	Sending mass emails, executing financial transactions, deleting data. Human must approve before the agent acts.
Low-Confidence Output	The agent's confidence score falls below a threshold; it routes to a human reviewer rather than guessing.
Edge Cases	Situations outside the agent's training distribution or that match known failure patterns.

Regulatory Compliance	Actions that require a human signature or audit trail for legal or compliance reasons.
Periodic Auditing	Even high-confidence autonomous flows benefit from random human sampling to detect gradual drift.

HITL Design Principles

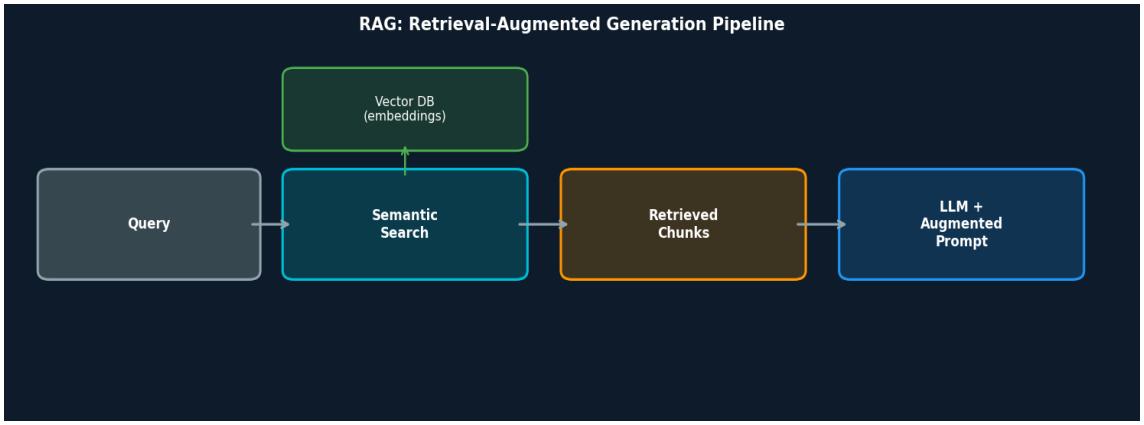
The goal is not to maximise human involvement (that defeats the purpose of an agent) but to place human judgment precisely where it adds the most value. Design principles:

- **Clarity**: When the agent asks for human input, present all relevant context clearly so the human can decide quickly and accurately.
- **Timeout Handling**: Define what the agent does if no human responds within a set time — safe fallback action or escalation.
- **Audit Trail**: Log every human intervention, the context provided, and the decision made.
- **Progressive Autonomy**: Start with high HITL involvement; as the agent proves reliable in specific scenarios, reduce oversight for those cases.

When to use	High-stakes domains (finance, healthcare, legal), new deployments before trust is established, edge cases and novel situations.
Key benefit	Trust and safety: human oversight prevents catastrophic autonomous errors. Regulatory compliance.
Key challenge	Human bottleneck: HITL can drastically slow throughput if triggered too frequently. Carefully calibrate thresholds.

Ch 14 Knowledge Retrieval (RAG)

An LLM's knowledge is static, bounded by its training cutoff and the general data it was trained on. It has no access to your company's internal documents, yesterday's news, or specialised domain knowledge. **Retrieval-Augmented Generation (RAG)** solves this by enabling the agent to **retrieve relevant information from an external knowledge base** at query time, then augment the LLM's prompt with this retrieved context before generating a response.



RAG pipeline: semantic search retrieves relevant document chunks to augment the LLM prompt.

Core Technical Concepts

Embeddings	Text is converted into high-dimensional numerical vectors (e.g. 1536 dimensions) that capture semantic meaning. Words or passages with similar meanings have vectors close together in this space. "Kitten" and "cat" are close; "car" is distant.

Vector Database	A specialised database (Pinecone, ChromaDB, Weaviate, pgvector) stores and indexes these embedding vectors for efficient similarity search at scale.
Semantic Search	Unlike keyword search, semantic search finds documents that are meaningfully similar to the query even if they use different words. Measured by cosine similarity or dot product of embeddings.
Chunking Strategy	Documents must be split into chunks before embedding. Chunk size affects retrieval quality: too small loses context, too large introduces noise. Optimal chunk size is domain-dependent (typically 256–1024 tokens).
Re-ranking	A second-stage model re-scores retrieved chunks for relevance to the query before passing to the LLM, improving precision beyond raw vector similarity.

Benefits of RAG

- **Up-to-date information**: The knowledge base is updated independently of the LLM, eliminating training cutoff issues.
- **Reduced hallucination**: Grounding responses in retrieved, verifiable documents dramatically reduces fabrication.
- **Domain specialisation**: Works with proprietary internal documents (policies, manuals, research) the LLM was never trained on.
- **Citations**: RAG enables agents to cite the exact source document for each fact — critical for trust and verifiability.

When to use	Enterprise knowledge search, legal and medical Q&A;, technical documentation assistance, customer support with access to product manuals.
Key benefit	Factual grounding: transforms the LLM from a fluent guesser into a fact-based responder.
Key challenge	Retrieval quality: if the wrong chunks are retrieved, the LLM still gives wrong answers (garbage in, garbage out). Invest in chunking, indexing, and re-ranking.

PART FOUR — Advanced Patterns (Chapters 15–21)

Part Four covers the frontier of agentic design: how agents communicate across framework boundaries, how they optimise resource usage, how they reason explicitly through complex problems, how to keep them safe at scale, how to measure their performance, how to prioritise under constraints, and how to discover the unknown.

Ch 15 Inter-Agent Communication (A2A)

As agentic ecosystems grow, agents built on different frameworks need to collaborate. Google's **Agent-to-Agent (A2A) protocol** is an open standard for enabling cross-framework agent communication. Like MCP standardises tool access, A2A standardises **how agents delegate tasks to other agents**, regardless of their internal implementation.

A2A vs MCP

MCP (Ch 10)	Standardises how an agent accesses tools and data sources (vertical integration: agent → tools).
A2A	Standardises how agents communicate with other agents (horizontal integration: agent ↔ agent).

A2A Key Concepts

Agent Card	A standardised JSON descriptor published by each agent: its capabilities, input/output schemas, and endpoint URL. Agents discover each other via these cards.
Task Delegation	Agent A discovers Agent B via its card, sends a task request conforming to B's input schema. B processes and returns a structured result.
Status Streaming	Long-running tasks return intermediate status updates, not just a final result. The delegating agent can monitor progress or cancel.
Interoperability	An agent built with LangChain can delegate to one built with Google ADK or Crew AI, as long as both implement A2A.
When to use	Multi-framework enterprise systems, agent marketplaces, cross-team collaboration where different teams own different agents built with different tools.
Key benefit	True interoperability: removes vendor lock-in and enables composition of the best agent for each sub-task.
Key challenge	Trust and authentication between agents: a malicious agent card could redirect tasks to harmful endpoints.

Ch 16 Resource-Aware Optimisation

Agents operating at scale consume significant computational resources: LLM API calls, tool invocations, memory retrievals. **Resource-Aware Optimisation** enables agents to **dynamically monitor and trade off** between cost, latency, and quality — choosing the cheapest option that still meets the quality bar.

Resource Dimensions

Computational Cost	Token cost per LLM call. Use smaller/cheaper models for simple routing tasks; reserve expensive models for complex reasoning.

Latency	Time to first token, total response time. Parallelise where possible (Ch 3). Cache frequent queries.
Token Budget	Hard limits on tokens per session or per task to control cost. The agent must prioritise which information fits within budget.
Quality vs Cost	The core trade-off: a 7B parameter model may handle 80% of queries adequately at 10% the cost of GPT-4. Route accordingly.
Environmental Cost	Energy consumption and carbon footprint. Increasingly relevant for sustainability-conscious deployments.

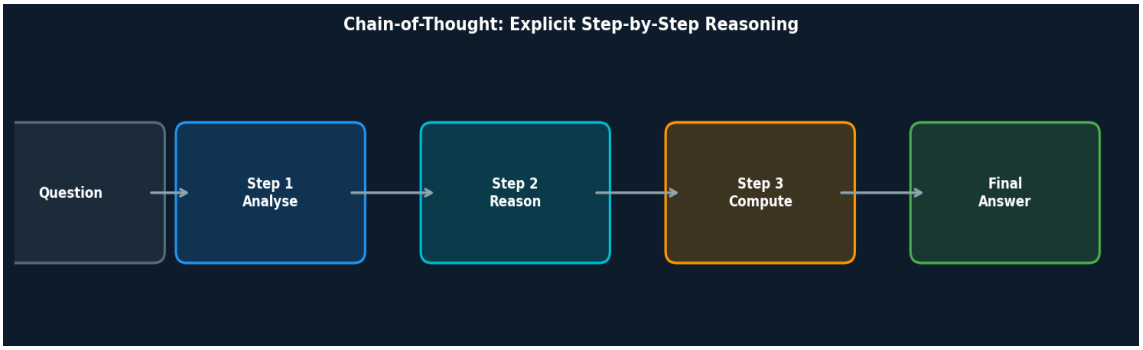
Optimisation Strategies

- **Model cascading**: Start with a small, fast model. Escalate to a larger model only when the small one's confidence falls below threshold.
- **Semantic caching**: Cache LLM responses for semantically similar queries. If a new query is close enough to a cached one, return the cached result.
- **Adaptive tool selection**: Choose cheaper tool implementations when latency or cost constraints are active.
- **Context compression**: Compress the context window to fewer tokens before expensive LLM calls.

When to use	High-volume production systems, budget-constrained deployments, latency-sensitive applications (real-time customer support, trading).
Key benefit	Cost efficiency: dramatic reduction in API spend without proportional quality degradation.
Key challenge	Quality assurance: ensure cheaper models genuinely meet quality thresholds for each task type. Monitor regressions.

Ch 17 Reasoning Techniques

Complex problems — multi-step mathematics, code debugging, strategic planning, medical diagnosis — require more than a quick LLM call. **Reasoning Techniques** are prompting and inference strategies that make the agent's thinking **explicit and deliberate**, dramatically improving accuracy on hard problems by allocating more computation to the reasoning process.



Chain-of-Thought: each intermediate reasoning step is explicit before the final answer.

Chain-of-Thought (CoT)

Chain-of-Thought prompting guides the LLM to generate intermediate reasoning steps before producing a final answer — "think step by step" — rather than jumping directly to a conclusion. This breaks complex problems into manageable sub-problems, dramatically improves accuracy on arithmetic, logic, and multi-hop reasoning, and makes the model's reasoning transparent and auditable.

Tree-of-Thought (ToT)

Tree-of-Thought extends CoT by exploring **multiple reasoning branches simultaneously**, forming a tree structure. Instead of committing to a single chain of reasoning, the agent considers several intermediate hypotheses at each step, evaluates them, prunes unpromising branches, and expands the most promising ones. This is especially powerful for problems with a large solution space (e.g. game-playing, mathematical proofs, architectural decisions).

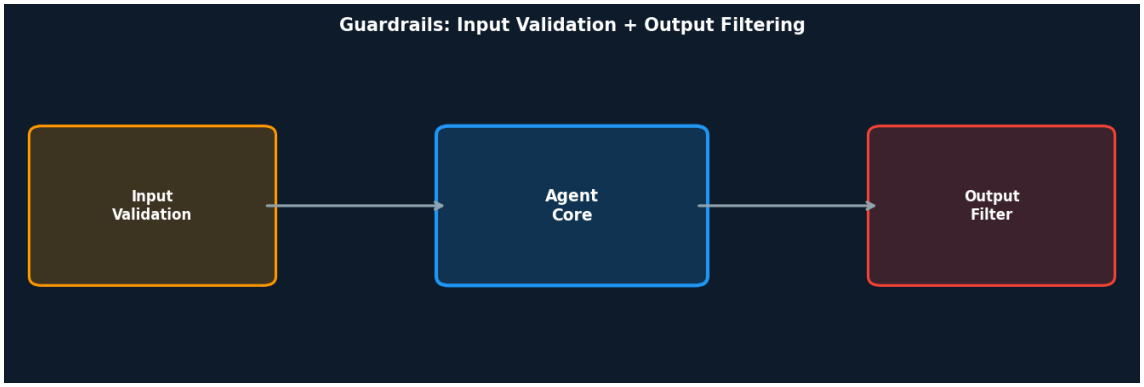
ReAct (Reasoning + Acting)

ReAct interleaves reasoning traces with tool calls: the agent alternates between "Thought" (internal reasoning about the current state and next action) and "Action" (calling a tool). The tool's result is an "Observation" that informs the next "Thought". This tight coupling of reasoning and real-world interaction makes ReAct the dominant pattern for tool-using agents that need to reason about dynamic, external information.

CoT vs ToT vs ReAct	CoT: single chain, fast, good for most problems. ToT: multi-branch exploration, expensive but powerful for hard combinatorial problems. ReAct: reasoning interleaved with tool use, ideal for agents with external tool access.
Extended Inference Time	A core principle of advanced reasoning: giving the LLM more "thinking time" (more tokens, more iterations) during inference significantly improves accuracy on hard problems. This is the basis of "thinking models" like o1 and Gemini Thinking.
When to use	Complex question answering, mathematical reasoning, code debugging, strategic planning, medical/legal analysis — any task where accuracy matters more than response speed.

Ch 18 Guardrails / Safety Patterns

Guardrails are the protective layer that ensures agents operate safely, ethically, and within intended boundaries — especially as they gain autonomy and access to powerful tools. They can be implemented at both the **input** (before the agent processes a request) and **output** (before results are returned to the user) stages.



Guardrails wrap the agent: input validation prevents harmful requests; output filtering prevents harmful responses.

Input Guardrails

Intent Detection	Classify user intent before processing. Reject or reroute requests that match harmful categories (prompt injection, jailbreaks, prohibited topics).
PII Detection	Identify and redact personally identifiable information before it enters the LLM context — critical for GDPR compliance.
Schema Validation	Validate that inputs conform to expected formats and value ranges before tool calls are made.

Rate Limiting	Prevent abuse by limiting the number of requests per user, session, or time window.

Output Guardrails

Toxicity Filtering	Scan agent output for harmful, offensive, or discriminatory content before returning to the user.
Fact Grounding	Cross-reference agent claims against the retrieved context (RAG). Flag or reject statements not supported by retrieved documents.
Action Validation	Before the agent executes an irreversible action (delete, send, purchase), validate against a ruleset or trigger HITL approval.
Output Schema	Enforce that structured outputs (JSON, function calls) conform to the expected schema before downstream systems consume them.
When to use	Always. Every deployed agent needs some form of guardrails. Critical in healthcare, finance, legal, and customer-facing systems.
Key benefit	Safety, ethics compliance, and regulatory readiness. Prevents the agent from causing harm even when the user tries to manipulate it.
Key challenge	Over-restriction: guardrails that are too aggressive block legitimate requests and frustrate users. Calibrate carefully with red-teaming.

Ch 19 Evaluation and Monitoring

An agent that cannot be measured cannot be trusted. **Evaluation and Monitoring** provides the systematic framework for assessing agent performance over time — not just at deployment, but continuously. While Goal Setting (Ch 11) defines objectives and Reflection (Ch 4) provides self-correction, this chapter focuses on **external, objective measurement** of effectiveness, efficiency, and compliance.

Evaluation Dimensions

Task Accuracy	Does the agent achieve the intended goal? Measured via golden datasets, human evaluation, or automated metrics (BLEU, ROUGE, exact match).
Reasoning Quality	Is the agent's reasoning process sound? Evaluated via CoT traces, logical consistency checks.
Latency & Throughput	How fast does the agent respond? How many tasks per second? Critical for real-time applications.
Cost Efficiency	Token cost per task. Helps identify expensive patterns and guide optimisation (connects to Ch 16).
Safety Compliance	Does the agent ever produce harmful, biased, or off-policy outputs? Measured via adversarial testing and red-teaming.
Drift Detection	Are there performance regressions over time as data distributions shift? Monitored via rolling evaluation windows.

Evaluation Methodologies

- **Golden dataset evaluation**: A curated set of input/expected-output pairs used as a benchmark. Run regularly to detect regressions.

● **A/B testing**: Run two agent versions in parallel; measure which performs better on real traffic with statistical significance.

● **LLM-as-judge**: Use a powerful LLM (e.g. GPT-4) to evaluate the quality of another agent's outputs — scalable but introduces its own biases.

● **Human evaluation**: Ground truth for subjective quality. Expensive but irreplaceable for high-stakes domains.

● **Observability**: Trace every LLM call, tool invocation, and decision point. Tools like LangSmith, OpenTelemetry, and Vertex AI provide this visibility.

When to use	Every production deployment. Set up evaluation pipelines before launch; run continuously post-launch.
Key benefit	Continuous quality assurance: catch regressions before they harm users. Build organisational trust in the agent.
Key challenge	Evaluation is expensive. Design efficient automated pipelines for frequent testing, reserve human evaluation for critical edge cases.

Ch 20 Prioritisation

Agents operating in complex environments face more tasks than they can handle with available resources. Without a principled approach to deciding what to do next, they become inefficient or miss critical objectives. The **Prioritisation** pattern gives agents the ability to rank tasks by urgency, importance, dependencies, and resource cost.

Prioritisation Frameworks

Urgency × Impact Matrix	Classify tasks on two axes: urgency (how time-sensitive?) and impact (how much value does completing this create?). Prioritise high-urgency, high-impact tasks first.
Dependency-Aware Ordering	Respect task dependencies: a task that blocks others has higher implicit priority than its standalone value suggests.
Resource-Constrained	Given a token budget or time limit, select the highest-value set of tasks that fits within constraints (a knapsack problem).
Dynamic Re-prioritisation	As the environment changes (new tasks arrive, resources are consumed, priorities shift), the agent re-ranks dynamically.
When to use	Multi-task agents, real-time systems, agents with explicit resource budgets, long-horizon autonomous agents managing many concurrent objectives.
Key benefit	Efficiency and goal alignment: limited resources are applied where they create the most value.
Key challenge	Priority inversion: low-priority tasks blocking high-priority ones. Deadline management: urgent tasks with hard deadlines must be identified.

Ch 21 Exploration and Discovery

Most agent patterns covered so far assume the agent knows what problem to solve. **Exploration and Discovery** addresses a different challenge: finding the unknown unknowns — opportunities, insights, and solutions that were not explicitly requested. This pattern is about giving agents the curiosity and tools to **proactively venture into unfamiliar territory**.

Exploration vs Exploitation

The core tension in exploration is the classic **exploration-exploitation trade-off**: should the agent use a known-good strategy (exploit) or try something new that might be better (explore)? Strategies for balancing this include:

ε-Greedy	With probability ε, choose a random action (explore); otherwise choose the best-known action (exploit). Simple but effective.
UCB (Upper Confidence Bound)	Choose actions that maximise a combination of expected reward and uncertainty. Naturally balances exploration and exploitation.
Curiosity-Driven	The agent receives an intrinsic reward for discovering novel states, independently of the task reward. Encourages broad exploration.
Hypothesis Testing	The agent generates hypotheses about the world, designs experiments to test them, and updates its model based on results.

Applications of Exploration

- **Scientific research agents**: Autonomously generate and test hypotheses in a defined domain (biology, materials science).
- **Market intelligence**: Discover emerging trends, competitor moves, or new market segments without explicit search queries.
- **Creative AI**: Generate novel solutions, designs, or content by systematically exploring possibility spaces.
- **Security red-teaming**: Explore an agent's or system's attack surface to discover vulnerabilities before adversaries do.

When to use	Scientific research, competitive intelligence, creative problem-solving, novel environment navigation.
Key benefit	Discovery of value beyond the explicitly requested: the agent finds things the user didn't know to ask for.
Key challenge	Bounding exploration: without constraints, an agent can explore indefinitely. Define scope boundaries, resource budgets, and safety guardrails.

Quick Reference — All 21 Patterns

#	Pattern	Core Idea
1	**Prompt Chaining**	Sequential LLM pipeline; divide-and-conquer for complex tasks
2	**Routing**	Dynamic intent-based dispatch to specialised sub-agents
3	**Parallelization**	Concurrent independent sub-tasks; fan-out / fan-in synthesis
4	**Reflection**	Self-evaluating feedback loop; Producer + Critic model
5	**Tool Use**	Function calling to extend agent beyond training data
6	**Planning**	Multi-step plan formulation before execution; adaptive
7	**Multi-Agent**	Specialised agent teams coordinated by an orchestrator

#	Pattern	Core Idea
8	**Memory Management**	Short-term (context) + long-term (vector DB) agent memory
9	**Learning & Adaptation**	PPO, DPO, few-shot, online learning for continuous improvement
10	**Model Context Protocol**	Universal open standard for LLM ↔ tool/data connectivity
11	**Goal Setting**	Explicit objectives, success criteria, and progress monitoring

#	Pattern	Core Idea
12	**Exception Handling**	Fault detection, recovery strategies, controlled degradation
13	**Human-in-the-Loop**	Strategic human oversight at critical decision points
14	**RAG**	Semantic retrieval from external knowledge bases; grounded responses

#	Pattern	Core Idea
15	**Inter-Agent (A2A)**	Cross-framework agent-to-agent task delegation via open protocol
16	**Resource-Aware**	Dynamic cost/latency/quality trade-offs; model cascading
17	**Reasoning (CoT/ToT/ReAct)**	Explicit step-by-step or multi-branch reasoning for hard problems
18	**Guardrails**	Input validation + output filtering for safe, ethical operation
19	**Evaluation**	Systematic performance measurement, A/B testing, drift detection
20	**Prioritisation**	Dynamic task ranking by urgency, impact, and resource constraints
21	**Exploration**	Proactive discovery of unknown opportunities and novel solutions

Key Takeaways

Patterns are not rigid rules
They are battle-tested templates. The real skill is recognising which pattern(s) apply to your problem and combining them effectively. A single production system may use Prompt Chaining, Reflection, Tool Use, RAG, and Guardrails simultaneously.

Master the core four first
Prompt Chaining, Routing, Tool Use, and Reflection form the bedrock of nearly every production agentic system. Get these right before exploring more complex patterns.

Safety is non-negotiable from day one
Guardrails, Exception Handling, and HITL are not optional extras. Design safety in from the start. An agent with broad tool access and no guardrails is a liability, not an asset.

Start simple, scale intentionally
Multi-agent systems are powerful but complex and expensive to debug. Begin with a single agent. Add agents only when you have hit clear capability ceilings that a single agent cannot overcome.

Context engineering is the new prompt engineering
At Level 2+, the highest-leverage skill is curating precisely what information enters the LLM's context at each step. Too much context causes confusion; too little causes errors.

Evaluate before you trust
Measure everything. An agent that cannot be evaluated cannot be safely deployed at scale. Build your evaluation pipeline before your production pipeline.

MCP and A2A are the foundation of future ecosystems
The emergence of open standards for tool access (MCP) and agent-to-agent communication (A2A) signals a shift toward a modular, interoperable agent ecosystem — much like microservices transformed monolithic software architecture.

Original book: *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems* · Antonio Gulli · Springer 2025 · amazon.com/dp/3032014018 · All royalties donated to Save the Children.